

Generated aidlc-docs/ Reference

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [generated-docs-reference.pdf](#)

When you run the AI-DLC workflow, all documentation artifacts are generated inside an `aidlc-docs/` directory at your workspace root. The exact files created depend on your project type (greenfield vs brownfield), complexity, and which stages the workflow executes or skips.

Below is the fully populated structure showing every possible file across all phases and stages. Conditional files are annotated with notes indicating when they appear.

```

aidlc-docs/
├─ aidlc-state.md # Workflow state tracker – proje
ct info, stage progress, current status
├─ audit.md # Complete audit trail – every u
ser input, AI response, and approval with timestamps
|
├─ inception/ # ● INCEPTION PHASE – determine
s WHAT to build and WHY
|   └─ plans/
|       └─ execution-plan.md # Workflow visualization and pha
se execution decisions (always created)
|           └─ story-generation-plan.md # Story development methodology
and questions (if User Stories executes)
|               └─ user-stories-assessment.md # Assessment of whether user sto
ries add value (if User Stories executes)
|                   └─ application-design-plan.md # Component and service design p
lan with questions (if Application Design executes)
|                       └─ unit-of-work-plan.md # System decomposition plan with
questions (if Units Generation executes)
|                           |
|                           └─ reverse-engineering/ # Created only for brownfield pr
objects (existing codebase detected)
|                               └─ business-overview.md # Business context, transaction
s, and dictionary
|                                   └─ architecture.md # System architecture diagrams,
component descriptions, data flow
|                                       └─ code-structure.md # Build system, key classes/modu
les, design patterns, file inventory
|                                           └─ api-documentation.md # REST APIs, internal APIs, and
data models
|                                               └─ component-inventory.md # Inventory of all packages by t
ype (application, infrastructure, shared, test)
|                                                   └─ technology-stack.md # Languages, frameworks, infrast
ructure, build tools, testing tools
|                                                       └─ dependencies.md # Internal and external dependen
cy graphs and relationships
|                                                           └─ code-quality-assessment.md # Test coverage, code quality in
dicators, technical debt, patterns
|                                                               └─ reverse-engineering-timestamp.md # Analysis metadata and artifact
checklist
|                           |
|                           └─ requirements/
|                               └─ requirements.md # Functional and non-functional
requirements with intent analysis (always created)
|                                   └─ requirement-verification-questions.md # Clarifying questions with [Ans
wer]: tags for user input (always created)
|                                       |
|                                       └─ user-stories/ # Created only if User Stories s
tage executes
|                                           └─ stories.md # User stories following INVEST
criteria with acceptance criteria
|                                               └─ personas.md # User archetypes, characteristi
cs, and persona-to-story mappings

```

```

| |
| | └─ application-design/ # Created only if Application De
sign and/or Units Generation execute
| |   └─ application-design.md # Consolidated design document
(if Application Design executes)
| |   └─ components.md # Component definitions, respons
ibilities, and interfaces
| |   └─ component-methods.md # Method signatures, purposes, a
nd input/output types
| |   └─ services.md # Service definitions, responsib
ilities, and orchestration patterns
| |   └─ component-dependency.md # Dependency matrix and communic
ation patterns between components
| |   └─ unit-of-work.md # Unit definitions and responsib
ilities (if Units Generation executes)
| |   └─ unit-of-work-dependency.md # Dependency matrix between unit
s (if Units Generation executes)
| |   └─ unit-of-work-story-map.md # Mapping of user stories to uni
ts (if Units Generation executes)
| |
| └─ construction/ # ● CONSTRUCTION PHASE — determ
ines HOW to build it
| | └─ plans/
| | | └─ {unit-name}-functional-design-plan.md # Business logic design plan wit
h questions (per unit, if Functional Design executes)
| | | └─ {unit-name}-nfr-requirements-plan.md # NFR assessment plan with quest
ions (per unit, if NFR Requirements executes)
| | | └─ {unit-name}-nfr-design-plan.md # NFR design patterns plan with
questions (per unit, if NFR Design executes)
| | | └─ {unit-name}-infrastructure-design-plan.md # Infrastructure mapping plan wi
th questions (per unit, if Infrastructure Design executes)
| | | └─ {unit-name}-code-generation-plan.md # Detailed code generation steps
with checkboxes (per unit, always created)
| |
| | └─ {unit-name}/ # Per-unit artifacts — one direc
tory per unit of work
| | | └─ functional-design/ # Created only if Functional Des
ign executes for this unit
| | | | └─ business-logic-model.md # Detailed business logic and al
gorithms
| | | | └─ business-rules.md # Business rules, validation log
ic, and constraints
| | | | └─ domain-entities.md # Domain models with entities an
d relationships
| | | | └─ frontend-components.md # UI component hierarchy, props,
state, interactions (if unit has frontend)
| | |
| | | └─ nfr-requirements/ # Created only if NFR Requiremen
ts executes for this unit
| | | | └─ nfr-requirements.md # Scalability, performance, avai
lability, security requirements
| | | | └─ tech-stack-decisions.md # Technology choices and rationa
le
| |
| |

```

```

| | | └─ nfr-design/                                # Created only if NFR Design exe
cutes for this unit
| | | | └─ nfr-design-patterns.md                    # Resilience, scalability, perfo
rmance, and security patterns
| | | | └─ logical-components.md                     # Logical infrastructure compone
nts (queues, caches, etc.)
| | |
| | | └─ infrastructure-design/                      # Created only if Infrastructure
Design executes for this unit
| | | | └─ infrastructure-design.md                  # Cloud service mappings and inf
rastructure components
| | | | └─ deployment-architecture.md                 # Deployment model, networking,
scaling configuration
| | | |
| | | | └─ code/                                     # Markdown summaries of generate
d code (always created per unit)
| | | | └─ *.md                                       # Code generation summaries (act
ual code goes to workspace root)
| | |
| | | └─ shared-infrastructure.md                     # Shared infrastructure across u
nits (if applicable)
| | |
| | | └─ build-and-test/                             # Always created after all units
complete code generation
| | | | └─ build-instructions.md                     # Prerequisites, build steps, tr
oubleshooting
| | | | └─ unit-test-instructions.md                  # Unit test execution commands a
nd expected results
| | | | └─ integration-test-instructions.md           # Integration test scenarios, se
tup, and execution
| | | | └─ performance-test-instructions.md           # Load/stress test configuration
and execution (if performance NFRs exist)
| | | | └─ contract-test-instructions.md              # API contract validation betwee
n services (if microservices)
| | | | └─ security-test-instructions.md              # Vulnerability scanning and sec
urity testing (if security NFRs exist)
| | | | └─ e2e-test-instructions.md                  # End-to-end user workflow testi
ng (if applicable)
| | | | └─ build-and-test-summary.md                  # Overall build status, test res
ults, and readiness assessment
| | |
| | | └─ operations/                                # 🟡 OPERATIONS PHASE – placehol
der for future expansion

```

Notes

- `{unit-name}` is replaced with the actual unit name (e.g., `api-service`, `frontend-app`, `data-processor`). For single-unit projects, there is typically one unit directory under `construction/`.

- For simpler single-unit projects, the model may simplify naming — for example, `construction/plans/code-generation-plan.md` instead of `construction/plans/{unit-name}-code-generation-plan.md`, or place `application-design.md` as a single consolidated file without the individual component files.
- The `build-and-test/` directory always includes `build-and-test-summary.md`. The individual instruction files (`build-instructions.md`, `unit-test-instructions.md`, `integration-test-instructions.md`, etc.) are generated based on project complexity and testing needs.
- Plans in `inception/plans/` and `construction/plans/` contain `[Answer]:` tags where users provide input, and `[ ]`/`[x]` checkboxes that track execution progress.
- Application code is never placed inside `aidlc-docs/` — it goes to the workspace root. Only markdown documentation lives here.
- The `audit.md` file is append-only and captures every interaction with ISO 8601 timestamps.
- The `aidlc-state.md` file tracks which stages have been completed, skipped, or are in progress, along with extension configuration.